Locks lock lockables

Wording for LWG 2363

Document #:	P2160R1
Date:	2020-11-13
Project:	Programming Language C++
Audience:	LWG

Reply-to: Tim Song <t.canens.cpp@gmail.com>

§ 1 Abstract

This paper provides wording to clean up 32.5.5 [thread.lock] and resolve [LWG2363].

§ 2 Revision history

— R1: Rebased wording onto [N4868]. Removed “open issues” section.

§ 3 Drafting notes

The original complaint of [LWG2363] (a nonexistent *SharedTimedMutex* named requirement) has since been editorially resolved, and since these requirements are only intended to be used for standard library types rather than user code, I don’t see a need to promote them to *CamelCased* named requirements as in the current PR.

However, the previous drafting did reveal additional issues:

1. the current WP says that `shared_lock<Mutex>` requires `Mutex` to meet the “shared mutex requirements 32.5.4.5 [thread.sharedtimedmutex.requirements]”; this is a mismatch and also seemingly makes `shared_lock<shared_mutex>` undefined behavior outright even if the user doesn’t call the timed wait functions.
2. the current wording for `shared_lock` appears to disallow user-defined shared mutex types, because it references our internal requirements. This is a clear defect.
3. There is a pervasive problem throughout 32.5.5 [thread.lock] that conflates the preconditions of lock operations with the preconditions of the underlying lockable operations, and also confuses lockables with mutexes. The locks operate on lockables, so it’s a category error to ask whether the lockable is a “recursive mutex” (not to mention that this term is never properly defined); we should just forward to the underlying lockable and let that operation do whatever it does, whether that’s properly recursive locking, throwing an exception, or impregnating someone’s nonexistent cat.

The wording below introduces new *Cpp17SharedLockable* and *Cpp17SharedTimedLockable* named requirements. I decided to add the *Cpp17* prefix because they deal with components added in C++17.

Because the existing *Cpp17MeowLockable* requirements are very explicit that they do not deal with the nature of any lock ownership, the same is true for the new requirements. As far as the lockable requirements are concerned, “shared” and “non-shared” locks are distinguished solely by the functions used to acquire them.

As discussed above, the wording removes most explicit preconditions on lock constructors that are of the form “the calling thread does not own the mutex”; when instantiated with types that do not support recursive locking (and consider such attempts undefined behavior), this precondition is implicitly imposed by the call to the locking functions the constructors are specified to perform.

The `adopt_lock_t` overloads retain their precondition that the lock has been acquired, but re-expressed in lockable terms. This is not strictly necessary - failure to lock results in a precondition violation when the unlocking occurs - but appears to be harmless and potentially permits early diagnosis.

§ 4 Wording

This wording is relative to [N4868].

1. Edit 32.2.5.1 [thread.req.lockable.general] p3 as indicated:

3 The standard library templates `unique_lock` (32.5.5.4 [thread.lock.unique]), `shared_lock` (32.5.5.5 [thread.lock.shared]), `scoped_lock` (32.5.5.3 [thread.lock.scoped]), `lock_guard` (32.5.5.2 [thread.lock.guard]), `lock`, `try_lock` (32.5.6 [thread.lock.algorithm]), and `condition_variable_any` (32.6.5 [thread.condition.condvarany]) all operate on user-supplied lockable objects. The *Cpp17BasicLockable* requirements, the *Cpp17Lockable* requirements, and the *Cpp17TimedLockable* requirements, the *Cpp17SharedLockable* requirements, and the *Cpp17SharedTimedLockable* requirements list the requirements imposed by these library types in order to acquire or release ownership of a lock by a given execution agent.

[*Note 3:* The nature of any lock ownership and any synchronization it entails are not part of these requirements. — *end note*]

2. Add the following paragraph at the end of 32.2.5.1 [thread.req.lockable.general]:

4 A lock on an object `m` is said to be a *non-shared lock* if it is acquired by a call to `lock`, `try_lock`, `try_lock_for`, or `try_lock_until` on `m`, and a *shared lock* if it is acquired by a call to `lock_shared`, `try_lock_shared`, `try_lock_shared_for`, or `try_lock_shared_until` on `m`.

[*Note ?:* Only the method of lock acquisition is considered; the nature of any lock ownership is not part of these definitions. — *end note*]

3. Edit 32.2.5.2 [thread.req.lockable.basic] p3 and p4 as indicated:

```
m.unlock()
```

3 *Preconditions:* The current execution agent holds a non-shared lock on `m` .

4 *Effects:* Releases a non-shared lock on `m` held by the current execution agent.

5 *Throws:* Nothing.

4. Add the following subclauses under 32.2.5 [thread.req.lockable]:

§ ?.?.?.? *Cpp17SharedLockable* requirements [thread.req.lockable.shared]

1 A type `L` meets the *Cpp17SharedLockable* requirements if the following expressions are well-formed, have the specified semantics, and the expression `m.try_lock_shared()` has type `bool` (`m` denotes a value of type `L`):

`m.lock_shared()`

2 *Effects:* Blocks until a lock can be acquired for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

`m.try_lock_shared()`

3 *Effects:* Attempts to acquire a lock for the current execution agent without blocking. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

4 *Returns*: `true` if the lock was acquired, `false` otherwise.

`m.unlock_shared()`

5 *Preconditions:* The current execution agent holds a shared lock on `m`.

6 *Effects:* Releases a shared lock on `m` held by the current execution agent.

7 *Throws:* Nothing.

§ ?.?.?.? *Cpp17SharedTimedLockable* requirements [thread.req.lockable.shared.timed]

1 A type `L` meets the *Cpp17SharedTimedLockable* requirements if it meets the *Cpp17SharedLockable* requirements, and the following expressions are well-formed, have type `bool`, and have the specified semantics (`m` denotes a value of type `L`, `rel_time` denotes a value of a specialization of `duration`, and `abs_time` denotes a value of a specialization of `time_point`):

`m.try_lock_shared_for(rel_time)`

2 *Effects:* Attempts to acquire a lock for the current execution agent within the relative timeout (32.2.4 [thread.req.timing]) specified by `rel_time`. The function will not return within the timeout specified by `rel_time` unless it has

obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

3 *Returns*: `true` if the lock was acquired, `false` otherwise.

```
m.try_lock_shared_until(abs_time)
```

4 *Effects:* Attempts to acquire a lock for the current execution agent before the absolute timeout (32.2.4 [thread.req.timing]) specified by `abs_time`. The function will not return before the timeout specified by `abs_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

5 *Returns*: `true` if the lock was acquired, `false` otherwise.

5. Convert 32.5.4.2.1 [thread.mutex.requirements.mutex.general] p2 into a note:

2 [*Note ?:* The mutex types meet the *Cpp17Lockable* requirements (32.2.5.3 [thread.req.lockable.req]). — *end note*]

6. Convert 32.5.4.3.1 [thread.timedmutex.requirements.general] p2 into a note:

2 [*Note ?:* The timed mutex types meet the *Cpp17TimedLockable* requirements (32.2.5.4 [thread.req.lockable.timed]). — *end note*]

7. Add a note after 32.5.4.4.1 [thread.sharedmutex.requirements.general] p1:

? [*Note ?:* The shared mutex types meet the *Cpp17SharedLockable* requirements (?.?.?.? [thread.req.lockable.shared]). — *end note*]

8. Add a note after 32.5.4.5.1 [thread.sharedtimedmutex.requirements.general] p1:

? [*Note ?:* The shared timed mutex types meet the *Cpp17SharedTimedLockable* requirements (?.?.?.? [thread.req.lockable.shared.timed]). — *end note*]

9. Edit 32.5.5.2 [thread.lock.guard] as indicated:

```
explicit lock_guard(mutex_type& m);
```

2 *Preconditions:* If `mutex_type` is not a recursive mutex, the calling thread does not own the mutex `m`.

3 *Effects:* Initializes `pm` with `m`. Calls `m.lock()`.

```
lock_guard(mutex_type& m, adopt_lock_t);
```

4 *Preconditions:* The calling thread owns the mutex `m` holds a non-shared lock on `m`.

5 *Effects:* Initializes `pm` with `m`.

6 *Throws:* Nothing.

```
~lock_guard();
```

7 *Effects:* As if by Equivalent to: `pm.unlock()`.

10. Edit 32.5.5.3 [thread.lock.scoped] as indicated:

```
explicit scoped_lock(MutexTypes&... m);
```

2 *Preconditions:* If a `MutexTypes` type is not a recursive mutex, the calling thread does not own the corresponding mutex element of `m`.

3 *Effects:* Initializes `pm` with `tie(m...)`. Then if `sizeof...(MutexTypes)` is `0`, no effects. Otherwise if `sizeof...(MutexTypes)` is `1`, then `m.lock()`. Otherwise, `lock(m...)`.

```
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
```

4 *Preconditions:* The calling thread owns all the mutexes in `m` holds a non-shared lock on each element of `m`.

5 *Effects:* Initializes `pm` with `tie(m...)`.

6 *Throws:* Nothing.

```
~scoped_lock();
```

7 *Effects:* For all `i` in `[0, sizeof...(MutexTypes))`, `get<i>(pm).unlock()`.

11. Edit 32.5.5.4.2 [thread.lock.unique.cons] as indicated:

```
explicit unique_lock(mutex_type& m);
```

2 *Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

3 *Effects:* Calls `m.lock()`.

4 *Postconditions:* `pm == addressof(m)` and `owns == true`.

```
unique_lock(mutex_type& m, defer_lock_t) noexcept;
```

5 Postconditions: `pm == addressof(m)`and `owns == false`.

```
unique_lock(mutex_type& m, try_to_lock_t);
```

6 *Preconditions:* The supplied `Mutex` type meets the *Cpp17Lockable* requirements (32.2.5.3 [thread.req.lockable.req]). If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

7 *Effects:* Calls `m.try_lock()`.

8 *Postconditions:* `pm == addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock()`.

```
unique_lock(mutex_type& m, adopt_lock_t);
```

9 *Preconditions:* The calling thread owns the mutex holds a non-shared lock on `m`.

10 *Postconditions:* `pm == addressof(m)` and `owns == true`.

11 *Throws:* Nothing.

```
template<class Clock, class Duration>
  unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>&
        abs_time);
```

12 *Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type meets the *Cpp17TimedLockable* requirements (32.2.5.4 [thread.req.lockable.timed]).

13 *Effects:* Calls `m.try_lock_until(abs_time)`.

14 *Postconditions:* `pm == addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_until(abs_time)`.

```
template<class Rep, class Period>
  unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
```

15 *Preconditions:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type meets the *Cpp17TimedLockable* requirements (32.2.5.4 [thread.req.lockable.timed]).

16 *Effects:* Calls `m.try_lock_for(rel_time)`.

17 *Postconditions:* `pm == addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_for(rel_time)`.

12. Edit 32.5.5.5.1 [thread.lock.shared.general] as indicated:

1 An object of type `shared_lock` controls the shared ownership of a lockable object within a scope. Shared ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `shared_lock` object. Objects of type `shared_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime (6.7.3 [basic.life]) of the `shared_lock` object. The supplied `Mutex` type shall meet the shared mutex *Cpp17SharedLockable* requirements (32.5.4.5 [thread.sharedtimedmutex.requirements] ?.?.?.? [thread.req.lockable.shared]).

2 [*Note 1:* `shared_lock<Mutex>` meets the *Cpp17Lockable* requirements (32.2.5.3 [thread.req.lockable.req]). If `Mutex` meets the *Cpp17SharedTimedLockable* requirements

(?.?.?.? [thread.req.lockable.shared.timed]), `shared_lock<Mutex>` also meets the *Cpp17TimedLockable* requirements (32.2.5.4 [thread.req.lockable.timed]). — *end note*]

13. Edit 32.5.5.5.2 [thread.lock.shared.cons] as indicated:

```
explicit shared_lock(mutex_type& m);
```

2 *Preconditions:* The calling thread does not own the mutex for any ownership mode.

3 *Effects:* Calls `m.lock_shared()`.

4 *Postconditions:* `pm == addressof(m)` and `owns == true`.

```
shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

5 *Postconditions:* `pm == addressof(m)` and `owns == false`.

```
shared_lock(mutex_type& m, try_to_lock_t);
```

6 *Preconditions:* The calling thread does not own the mutex for any ownership mode.

7 *Effects:* Calls `m.try_lock_shared()`.

8 *Postconditions:* `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared()`.

```
shared_lock(mutex_type& m, adopt_lock_t);
```

9 *Preconditions:* The calling thread has shared ownership of the mutex. holds a shared lock on `m`.

10 *Postconditions:* `pm == addressof(m)` and `owns == true`.

```
template<class Clock, class Duration>
  shared_lock(mutex_type& m,
              const chrono::time_point<Clock, Duration>& abs_time);
```

11 *Preconditions:* The calling thread does not own the mutex for any ownership mode. `Mutex` meets the *Cpp17SharedTimedLockable* requirements (?.?.?.? [thread.req.lockable.shared.timed]).

12 *Effects:* Calls `m.try_lock_shared_until(abs_time)`.

13 *Postconditions:* `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_until(abs_time)`.

```
template<class Rep, class Period>
  shared_lock(mutex_type& m,
              const chrono::duration<Rep, Period>& rel_time);
```

14 *Preconditions:* The calling thread does not own the mutex for any ownership mode. `Mutex` meets the *Cpp17SharedTimedLockable* requirements (?.?.?.? [thread.req.lockable.shared.timed]).

15 *Effects:* Calls `m.try_lock_shared_for(rel_time)`.

16 *Postconditions:* `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_for(rel_time)`.

14. Edit 32.5.5.5.3 [thread.lock.shared.locking] as indicated:

```
template<class Clock, class Duration>
  bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
```

9 ¾ *Preconditions*: `Mutex` meets the *Cpp17SharedTimedLockable* requirements (?.?.?.? [thread.req.lockable.shared.timed]).

10 *Effects:* As if by `pm->try_lock_shared_until(abs_time)`.

11 *Returns:* The value returned by the call to `pm->try_lock_shared_until(abs_time)`.

12 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_until(abs_time)`.

13 *Throws*: Any exception thrown by `pm->try_lock_shared_until(abs_time)`. `system_error` when an exception is required (32.2.2 [thread.req.exception]).

14 *Error conditions:*

(14.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(14.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Rep, class Period>
  bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

14 ½ *Preconditions*: `Mutex` meets the *Cpp17SharedTimedLockable* requirements (?.?.?.? [thread.req.lockable.shared.timed]).

15 *Effects:* As if by `pm->try_lock_shared_for(rel_time)`.

16 *Returns:* The value returned by the call to `pm->try_lock_shared_for(rel_time)`.

17 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_for(rel_time)`.

18 *Throws*: Any exception thrown by `pm->try_lock_shared_for(rel_time)`. `system_error` when an exception is required (32.2.2 [thread.req.exception]).

19 *Error conditions:*

(19.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(19.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

§ 5 References

[LWG2363] Richard Smith. Defect in 30.4.1.4.1 [thread.sharedtimedmutex.class]. https://wg21.link/lwg2363

[N4868] Richard Smith. 2020. Working Draft, Standard for Programming Language C++. https://wg21.link/n4868